

Coding & Solution

Date	29 June 2026
Team ID	
Project Name	Rising Waters-ML Flood Prediction System
Maximum Marks	5 Marks

Coding & Solution

Field	Details
Repository Link / URL	chandeewarimothukoori-commits/Flood-Prediction-System
Programming Language(s)	Python 3.8+, HTML5, CSS3, JavaScript
Framework(s) Used	Flask, Scikit-learn, XGBoost, NumPy, Pandas, Matplotlib, Seaborn, Joblib, Jupyter Notebook
Key Features Implemented	• Data preprocessing (cleaning, feature scaling and train-test splitting). • Multiple ML models (Decision Tree, Random Forest, KNN and XGBoost). • Model evaluation using Accuracy Score, Confusion Matrix and Classification Report. • Trained XGBoost model saved using Joblib. • Flask-based web application for flood prediction. • User-friendly web interface with prediction input page. • Separate result pages for Flood Chance and No Flood Chance. • Responsive frontend using HTML, CSS and JavaScript. • Organized project structure with separate folders for templates, static files, models and dataset.
Pending / Incomplete Features	• IBM Cloud deployment. • Real-time weather API integration. • Database for prediction history. • Interactive dashboard with live flood monitoring. • SMS/Email notification for flood alerts.
Setup / Run Instructions	1. Install Python 3.8 or above. 2. Install required libraries using <code>pip install -r requirements.txt</code> (or install Flask, NumPy, Pandas, Scikit-learn, XGBoost, Joblib, Matplotlib and Seaborn). 3. Place the trained model (<code>floods.save</code>) and transformer (<code>transform.save</code>) inside the Notebook/Models folder. 4. Open the project folder in Visual Studio Code or Jupyter Notebook.

	5. Run <code>python app.py</code> . 6. Open the Flask application in a web browser using the local server URL (e.g., <code>http://127.0.0.1:5000</code>). 7. Enter rainfall and weather parameters to obtain flood prediction results.
--	---

Code Quality Checklist

S.No	Criteria	Status (Yes/No)
1	Code is modular and organized into functions/classes	Yes
2	Meaningful variable and function names	Yes
3	Comments/documentation	Partial
4	Error handling	Partial
5	Application runs without critical errors	Yes
6	Version control repository	Yes

Additional Notes / Comments

- The project follows a modular Flask architecture by separating frontend templates, static resources, machine learning models and datasets.
- The XGBoost model is selected as the final prediction model after comparing multiple classification algorithms.
- HTML, CSS and JavaScript are separated to improve readability, maintainability and code reusability.
- The application provides an intuitive web interface for entering weather parameters and viewing flood prediction results.
- The project is designed to be scalable and can be extended with real-time weather APIs, cloud deployment on IBM Cloud and advanced disaster management features in future versions.